A Complete Multiagent Framework for Robust and Adaptable Dynamic Job Shop Scheduling

Ning Liu, *Member, IEEE*, and Mohamed A. Abdelrahman, and Srini Ramaswamy, *Senior Member, IEEE*

***Abstract*—This paper presents a complete multiagent framework for dynamic job shop scheduling, with an emphasis on robustness and adaptability. It provides both a theoretical basis and some experimental justifications for such a framework: a job dispatching procedure for a completely reactive scheduling approach, combining real-time and predictive decision making. It resolves various disruptions as flexibly as dispatching rules while providing more stability. It is ready to be implemented in a distributed environment where agents have minimum global information thereby improving system fault tolerance. Computational experiments on dynamic job arrivals provide the experimental justification of the framework. First, a comparison of computational results on unpredictable job arrivals among the presented framework and commonly used dispatching rules is presented to show the effectiveness and robustness of the developed framework. Then, a comparison of the computational results among four cases of dynamic job arrivals is presented to demonstrate the effects of making full use of available uncertain information about disruptions using this framework for the enhancement of scheduling robustness.**

***Index Terms*—Dynamic job shop scheduling, multiagent systems, scheduling robustness.**

I. Introduction

JOB SHOP scheduling can be divided into two classes, namely, static and dynamic job shop scheduling. Static scheduling usually produces schedules in advance in order to direct operations and resource allocation. Unfortunately, in a dynamic environment, as soon as the schedule is released, it is immediately subject to random disruptions that may render the initial schedule obsolete and result in degraded system performance [1].

Previous approaches to scheduling in the presence of disruptions can be broadly classified into two groups. One group proposes job dispatching, which is completely reactive scheduling [2]; another group proposes control strategies that guide recovery of the system from disruptions with consideration of a precomputed schedule, such as predictive scheduling with rescheduling [3]–[6], robust scheduling [1], [7], [8], [9], predictable scheduling [10], [11], and knowledge-based scheduling [12], [13]. In completely reactive scheduling, no schedule is generated in advance and decisions are made locally in real time. A frequently used reactive approach is priority dispatching rules, which dispatches jobs dynamically to an available machine according to their priorities to account for random disruptions as they occur. While priority dispatching rules are efficient, they can bring poor long-term performance due to their myopic focus on local decisions [14].

All of these traditional approaches, however, are based on centralized models, in which a central computing unit performs all computations and views jobs and machines as passive symbols and not as active entities (as considered in certain decentralized models). Therefore, centralized approaches are inflexible, expensive, and slow to satisfy scheduling problems under disruptions [15]. One of the advantages of a certain type of agent-based distributed scheduling is its ability to effectively handle the dynamic nature of the job shop. Agents directly represent physical objects, such as machines, jobs, operators, etc., and they operate in a distributed architecture. Thus, this type of agent-based distributed manufacturing scheduling can realize dynamic scheduling effectively and provide high fault tolerance.

Previous approaches of this type of agent-based distributed manufacturing scheduling are based mainly on the combination of market-based mechanisms used by the multiagent systems, such as contract net protocols [16], [17] or auctions, with local decision schemes used by agents, such as dispatching rules [18]–[21], constraint satisfaction [22]–[24], and decision trees. Recently, research in this area has evolved to consider the combination of auctions with Lagrangian relaxation [26]–[29]. This progress provides the theoretical basis for bid computation, bid evaluation, and price adjustment; considers optimization during distributed problem solving; and overcomes system performance unpredictability and poor quality of schedules as in the earlier approaches [26]. Auction with Lagrangian relaxation approaches have been used in a quasi-distributed static job shop environment [27] and a holonic system [28]. A variant has been used in a distributed static job shop environment [26] and a single machine with dynamic job arrivals [29]. All of these previous techniques, however, pertain to job shop scheduling in static environments.

The goal of our research is to explore a methodology that uses distributed problem solvers—multiple agents—to effectively and efficiently schedule job shops in a dynamic environment with minimum global information and no master/slave relationships among the agents, and considers robustness and adaptability. By robustness, it is meant that the performance of the schedule remains satisfactory in the presence of disruptions. By adaptability, it is meant that the scheduling system is capable of generating schedules that accommodate a wide range of different, even unforeseen and unidentified disruptions.

Manuscript received June 1, 2005; revised March 27, 2006. This paper was recommended by Associate Editor L. Sheremetor.

N. Liu and M. A. Abdelrahman are with the Department of Electrical and Computer Engineering, Tennessee Technological University, Cookeville, TN 38505 USA (e-mail: nliu21@tntech.edu; mabdelrahman@tntech.edu).

S. Ramaswamy is with the Department of Computer Science, University of Arkansas at Little Rock, Little Rock, AR 72204 USA (e-mail: srini@acm.org; srini@ieee.org).

Digital Object Identifier 10.1109/TSMCC.2007.900658

1094-6977/$25.00 © 2007 IEEE

This research combines the variant of auction with Lagrangian relaxation approaches mentioned earlier [26] with a rolling time horizon procedure to form a complete multiagent framework (CMAF) for dynamic job shop scheduling. Rolling time horizon procedures have been applied to a wide range of dynamic optimization problems [30]. These procedures optimize the system in a limited time horizon instead of globally optimizing the system. Using a constant time horizon in auction with Lagrangian relaxation for dynamic job shop scheduling is inappropriate and incomplete. Our approach (i.e., CMAF) is based on a rolling time horizon formulation of auction with Lagrangian relaxation. Hence, we refer to this approach as a "complete" multiagent framework for dynamic job shop scheduling.

In this paper, we provide a theoretical basis and experimental verification of CMAF. The developed framework is a completely reactive scheduling approach combining real-time decision-making with predictive decision-making and can resolve various disruptions as flexibly as dispatching rules while providing a more stable response behavior.

The rest of this paper is organized as follows. Section II presents the theoretical basis of the framework providing the mathematical formulation of job shop scheduling and the resolution of its Lagrangian relaxation problem. Section III describes the complete multiagent framework in detail. Section IV provides solution schemes for dynamic job arrivals. Section V gives experimental design for dynamic job arrivals and Section VI gives its computational results and analysis. Section VII concludes the paper.

II. Problem Formulation and Resolution

The problem formulation is based on the discrete time, integer programming, and deterministic job shop scheduling formulation in [26], appropriately extended to include a rolling time horizon.

First, the following variables are defined, where operation j of job i is referred to as operation (i, j).

C_i	Completion time of operation (i, O_i).
d_i	Due date of job i.
INF	Positive infinity.
K_i	Start number of unprocessed operations of job i.
L	Length of rolling time horizon.
LB^r	Lower bound of iteration r of the Lagrangian relaxation problem.
M_c	Current total number of machine agents.
N_c	Current total number of job agents.
O_i	Total number of operations of job i.
p_{ij}	Processing time of operation (i, j).
r_i	Release date of job i.
S^r	Step size of iteration r.
SG^r_{mt}	Subgradient of iteration r.
t	Time-slot index.
t_c	Time slot of current decision point.
T_i	Tardiness of job $i, T_i = \max\{0, C_i - d_i\}$.
UB^r	Upper bound of iteration r of the Lagrangian relaxation problem.
w_i	Weight of job i.
X_{ijt}	0–1 Interger variable equals 1 if operation (i, j) is completed at time t.
Y_{ijm}	0–1 Integer variable equals 1 if operation (i, j) is processed on machine m.
α^r	Subgradient multiplier of iteration $r, 0 < \alpha^r \leq 2$.
λ_{mt}	Lagrange multiplier.
λ^r	The matrix of Lagrange multipliers of iteration r.

Then, we introduce three varying variables N_c, M_c, and $t_c + L$. N_c represents a varying number of job agents, M_c the varying number of machine agents, and $t_c + L$ represents a rolling time horizon. The integer programming formulation for the dynamic job shop problem with total weighted tardiness objective is defined as follows:

$$(P) \qquad \min \sum_{i=1}^{N_c} \sum_{t=t_c}^{t_c+L} w_i T_i X_{iO_i t}$$

subject to

$$\sum_{t=t_c}^{t_c+L} X_{ijt} = 1, \quad \forall i, j \tag{1}$$

$$\sum_{t=t_c}^{t_c+L} tX_{ijt} + p_{i,j+1} \leq \sum_{t=t_c}^{t_c+L} tX_{i,j+1,t}, \quad \forall i, j \tag{2}$$

$$\sum_{i=1}^{N_c} \sum_{j=K_i}^{O_i} X_{ijt} Y_{ijm} + \sum_{i=1}^{N_c} \sum_{j=K_i}^{O_i} \sum_{t'=t+1}^{\min\{L, t+p_{ij}-1\}} \times X_{ijt'} Y_{ijm} \leq 1, \quad \forall m, t \tag{3}$$

$$\sum_{t=t_c}^{t_c+L} tX_{i1t} \geq p_{i1} + r_i, \quad \forall i \tag{4}$$

$$X_{ijt} \in \{0,\ 1\}, \quad \forall i, j, t$$

$$Y_{ijm} \in \{0,\ 1\}, \quad \forall i, j, m$$

where $i = 1, \ldots N_c; j = 1, \ldots, O_i; m = 1, \ldots, M_c$ and $t = t_c, \ldots, t_c + L$.

The constraints described in (1) ensure that each operation of each job can only be completed in one time slot. The constraints described in (2) are operation precedence constraints for each job that ensure that the completion times of two consecutive operations are separated by the processing time of the later operation. The constraints described in (3) are machine capacity constraints that ensure that the capacity of a machine is not violated for its each time slot. The constraints described in (4) are job release date constraints that ensure that the first operation cannot be completed before the job has been in the shop for at least the time equal to the processing of the first operation.

When the machine capacity constraints (3) are relaxed by using Lagrange multipliers ($\lambda_{mt} (m = 1, \ldots, M_c; t = t_c, \ldots, t_c + L)$, the Lagrangian relaxation problem shown near the bottom of this page is obtained, subject to (1), (2), and (4).

The relaxed problem can be decomposed into independent job-level subproblems, shown at the bottom of this page for the given set of multipliers subject to

$$\sum_{t=t_c}^{t_c+L} X_{ijt} = 1, \quad \forall j \tag{5}$$

$$\sum_{t=t_c}^{t_c+L} tX_{ijt} + p_{i,j+1} \le \sum_{t=t_c}^{t_c+L} tX_{i,j+1,t}, \quad \forall j \tag{6}$$

$$\sum_{t=t_c}^{t_c+L} tX_{i1t} \ge p_{i1} + r_i \tag{7}$$

$$X_{ijt} \in \{0,\ 1\}, \quad \forall j, t$$

$$Y_{ijm} \in \{0,\ 1\}, \quad \forall j, m$$

where $i = 1, \ldots, N_c; j = 1, \ldots, O_i; m = 1, \ldots, M_c$; and $t = t_c, \ldots, t_c + L$.

Then, the dual problem is obtained as follows, where $V(P_{\lambda_i})$ denotes the value of optimal solution of problem (P_{λ_i}):

$$(P_D) \qquad \max \left\{ \min \left[\sum_{i=1}^{N_c} V(P_{\lambda_i}) - \sum_{t=t_c}^{t_c+L} \sum_{m=1}^{M_c} \lambda_{mt} \right] \right\}$$

subject to $\lambda \ge 0$ where $(\lambda = (\lambda_{mt}]; m = 1, \ldots, M_c$; and $t = t_c, \ldots, t_c + L$.

The dual problem can be solved by the subgradient search algorithm, which iteratively determines optimal λ_{mt} given X_{ijt} according to the following formulas (F_S):

$$(F_s)\lambda_{mt}^{r+1} = \max\{0, \lambda_{mt}^r + S^r \cdot \mathrm{SG}_{mt}^r\} \tag{8}$$

$$\mathrm{SG}_{mt}^r = \left\{ \sum_{i=1}^{N_c} \sum_{j=K_i}^{O_i} X_{ijt} Y_{ijm} + \sum_{i=1}^{N_c} \sum_{j=K_i}^{O_i} \sum_{t'=t+1}^{\min\{L, t+p_{ij}-1\}} X_{ijt} Y_{ijm} - 1 \right\} \tag{9}$$

$$S^r = \frac{\alpha^r \cdot (\mathrm{UB}^r - \mathrm{LB}^r)}{\sum_{m=1}^{M_c} \sum_{t=t_c}^{t_c+L} (\mathrm{SG}_{mt}^r)^2} \tag{10}$$

$$\mathrm{LB}^r = \max\left\{\mathrm{LB}^{r-1},\ \lceil V(P_D(\lambda^r))\rceil\right\} \tag{11}$$

$$\mathrm{UB}^r = \min\left\{\mathrm{UB}^{r-1},\ V\left(P\left(\lambda_{\mathrm{feasible}}^r\right)\right)\right\} \tag{12}$$

$$\mathrm{LB}^0 = 0 \tag{13}$$

$$\mathrm{UB}^0 = \mathrm{INF} \tag{14}$$

where $m = 1, \ldots, M_c; \quad t = t_c, \ldots, t_c + L. \quad V(P_D(\lambda^r))$ denotes the value of problem (P_D) with $\lambda^r . V(P(\lambda_{\mathrm{feasible}}^r))$ denotes the value of problem (P) with (λ^r and its feasible solutions. Stopping criteria are $UB^r - LB^r < \varepsilon$, $(\varepsilon > 0; \alpha^r$ becomes very small; or $r >$ maximum number of iterations.

III. Complete Multiagent Framework

When a Lagrange multiplier λ_{mt} denotes the price of using a time-slot t on machine m, the subgradient search procedure can be transformed into a combinatorial auction, in which bidders demand a set (or a combination) of objects with a single bid. In such a combinatorial auction, each job and each machine is an agent; each job is also a bidder; and a bidder demands a set of time slots with a single bid. An auctioneer proposes prices for using each time slot of machines with the objective of maximizing their profits; and bidders construct their bids of the wanted time slots with the objective of minimizing their costs.

A. System Architecture

An auction with Lagrangian relaxation approach can have two very different types of multiagent system architectures (Fig. 1). In the architecture on the left, there is only one auctioneer representing all machine agents, in which scheduling can only be considered in a static environment. Such an architecture is used in [27] and [28]. In the architecture on the right, there are multiple auctioneers; every machine agent becomes an auctioneer when it needs. In this case, scheduling can be done in a dynamic environment. Such an architecture is used in [26]. This is a real decentralized market organization [31]. The CMAF presented herein is based on this architecture.

The common shortcoming of auction with Lagrangian relaxation approaches is the use of a constant time horizon. The length of time horizon has to be long enough to cover the whole problem size while not to make the problem computationally intractable. Increasing the length of time horizon increases computational times and it limits the approaches' application to only static environments.

$$(P_\lambda) \quad \min \sum_{i=1}^{N_c} \sum_{t=t_c}^{t_c+L} \left[w_i T_i X_{iO_i t} + \sum_{m=1}^{M_c} \lambda_{mt} \sum_{j=K_i}^{O_i} \left(X_{ijt} Y_{ijm} + \sum_{t'=t+1}^{\min\{L, t+p_{ij}-1\}} X_{ijt'} Y_{ijm} \right) \right] - \sum_{t=t_c}^{t_c+L} \sum_{m=1}^{M_c} \lambda_{mt}$$

$$(P_{\lambda_i}) \qquad \min \sum_{t=t_c}^{t_c+L} \left[w_i T_i X_{iO_i t} + \sum_{m=1}^{M_c} \lambda_{mt} \sum_{j=K_i}^{O_i} \left(X_{ijt} Y_{ijm} + \sum_{t'=t+1}^{\min\{L, t+p_{ij}-1\}} X_{ijt'} Y_{ijm} \right) \right]$$

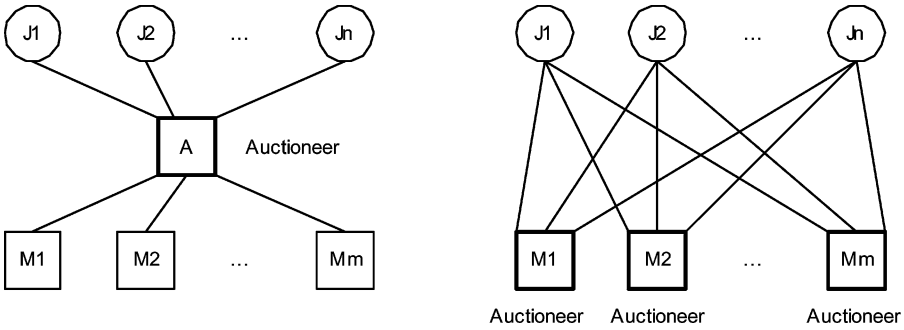


Fig. 1. System architecture.

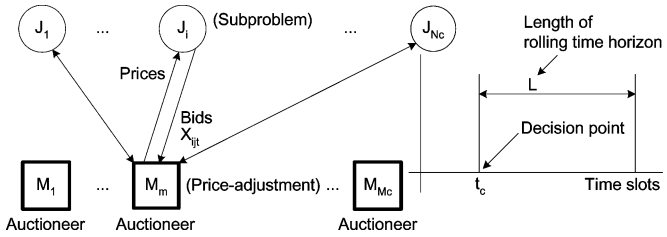


Fig. 2. CMAF.

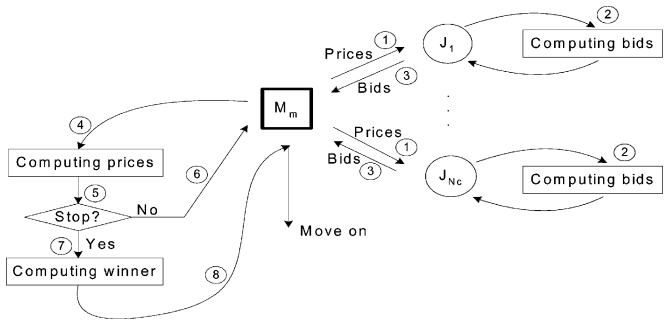


Fig. 3. Flow chart of CMAF scheduling mechanism.

B. System Mechanism

Fig. 2 shows a schematic for CMAF where auction with Lagrangian relaxation is based on a rolling time horizon procedure and a purely distributed architecture for dynamic job shop scheduling. The mechanism for CMAF operation is discussed hereafter. The description is better understood by reference to Fig. 2.

In CMAF, each job and each machine is an agent. Each machine agent is also an auctioneer and each job agent is a bidder. When a machine becomes available, it is at decision-point t_c to select an operation of a job to process. At every decision-point t_c, the machine agent becomes an auctioneer and hosts a combinatorial auction with proposed prices λ for using each time slot of all the machines. All bidders construct their bids for the wanted time slots according to their local decision-making schemes (*Subproblem*), i.e., $(P_{\lambda i})$. Coordination of bidders and the auctioneer is achieved through the iterative adjustment of these prices according to (*Price-adjustment*), i.e., (F_S). At the termination of such price-adjusting iterations, the auctioneer selects a bid having t_c as a start time; otherwise, it waits for the next decision point to start another auction or uses one of the dispatching rules to select an operation if there is an available operation needing to be processed on this machine. Thus, the auctioneer always gets a solution in every auction if there is an available operation.

Fig. 3 is a flow chart of the CMAF scheduling mechanism that is further summarized as follows.

Step 1) Machine agent, also an auctioneer, starts an auction with prices $\lambda(\lambda = [\lambda_{mt}]; m = 1, \ldots, M_c;$ and $t = t_c, \ldots, t_c + L)$ of using each time slot of all the machines.

Step 2) Each job agent, also a bidder, computes its bids of the wanted time slots $X_{ijt}(i = 1, \ldots, N_c; j = K_i, \ldots, O_i; m = 1, \ldots, M_c;$ and $t = t_c, \ldots, t_c + L)$ according to (*Subproblem*), i.e., $(P_{\lambda i})(i = 1, \ldots, N_c)$.

Step 3) Each job agent submits its bids $X_{ijt}(i = 1, \ldots, N_c; j = K_i, \ldots, O_i; m = 1, \ldots, M_c;$ and $t = t_c, \ldots, t_c + L)$ to the auctioneer.

Step 4) The auctioneer accepts all the bids and computes the prices according to (*Price-adjustment*), i.e., (F_S).

Step 5) The auctioneer checks if a stopping criterion is satisfied.

Step 6) If no, the auctioneer starts the next iteration and goes to Step 1 with new prices.

Step 7) If yes, the auctioneer computes a bidding winner. The auctioneer selects a bid having t_c as a start time; otherwise, uses one of the dispatching rules to select an operation, if there is an available operation that needs to be processed on this machine.

Step 8) The machine agent informs the winning job agent and processes the selected operation or waits for the next decision point to start another auction.

C. System Characteristics

According to the definition about dispatching in [2], "dispatching, in the production scheduling context, is a procedure that uses logical decision rules to select a job for processing on a machine that has just become available." Thus, the CMAF framework is actually a job dispatching procedure, a completely reactive scheduling approach. In CMAF, the logical decision rule is the auction. The solution of one auction provides a ranking of operations processed on a machine so that every machine agent can construct its schedule locally and in real time, and at the same time have predictive decision-making of time horizon

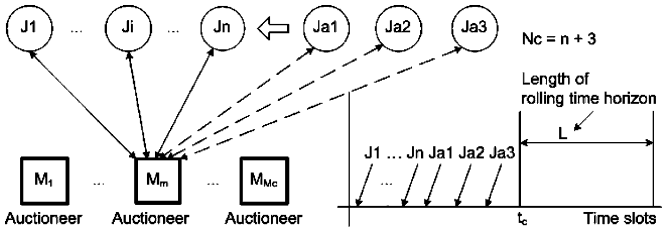


Fig. 4. Unpredictable job arrivals.

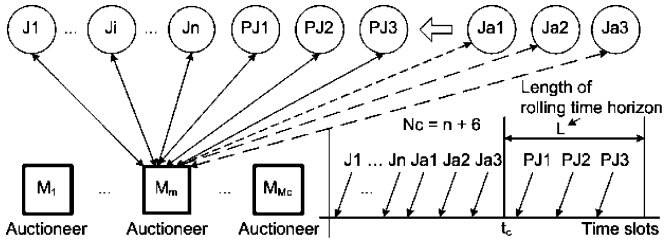


Fig. 5. Perfect forecast of job arrivals.

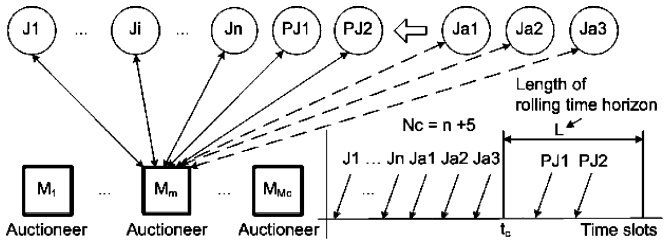


Fig. 6. Partial perfect forecast of job arrivals.

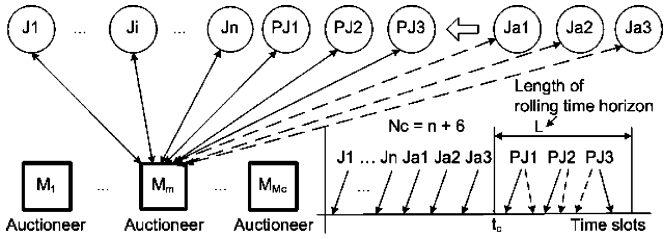


Fig. 7. Good forecast of job arrivals.

L based on mathematical programming. All machine agent auctions form simultaneous and repeated combinatorial auctions. Thus, it combines real-time decision-making with predictive decision-making based on optimization.

As the system architecture is purely distributed, the framework inherits the advantages of decentralized models, such as flexibility and high fault tolerance.

The advantage of this framework is its ability to handle the dynamic nature of job shop quite effectively. A new job arrival for processing within the given time horizon implies that the job agent can just start participating in the next auctions announced by the machine agents. Processing time variations mean that the machine agent advances or postpones hosting auctions; a machine breakdown means that this machine agent stops hosting auctions until the time it is repaired; if rerouting is allowed for job agents, they can begin bidding on their second-choice machine when the first-choice machine fails. Under the framework, agents can enter or exit the scheduling system flexibly without disturbing other agents so that various unpredictable disruptions can be resolved as flexibly as dispatching rules, thereby achieving scheduling robustness and adaptability.

IV. Solution Schemes for Dynamic Job Arrivals

This paper provides an experimental justification of the arguments presented in Section III-C using computational experiments on dynamic job arrivals. Four solution schemes for dynamic job arrivals are proposed: unpredictable job arrivals, perfect forecast of job arrivals, partial perfect forecast of job arrivals, and good forecast of job arrivals. The idea behind the use of these different schemes is the testing of the framework under a varied amount of available information regarding the predictability of the times of job arrivals so as to see the effects of making full use of available uncertain information of disruptions on the framework for further enhancement of scheduling robustness. In the sequel, we define and discuss the definitions of these different scenarios for dynamic job arrivals.

A. Unpredictable Job Arrivals

In unpredictable job arrivals' case (Fig. 4), all job arrivals are unpredictable. n is the number of already existing jobs. Already existing jobs J_i and just-arrived jobs J_{ai} form job agents. All of them participate in auctions beginning at decision-point t_c.

B. Perfect Forecast of Job Arrivals

In perfect forecast of job arrivals' case (Fig. 5), all job arrivals are exactly known in advance, i.e., predictable. Already existing jobs J_i and just-arrived jobs J_{ai} form job agents. Jobs PJ_i with perfect forecast of their arrivals within time horizon L form pseudo-job-agent. All of the job agents and pseudo-job-agent participate in auctions beginning at decision-point t_c. pseudo-job-agent are also job agents. However, they represent jobs that have not arrived at the current decision point, but are forecasted to arrive within the given time horizon. These pseudo-job-agent can participate in following auctions to make predictive decision-making better.

C. Partial Perfect Forecast of Job Arrivals

In partial perfect forecast of job arrivals' case (Fig. 6), part of the job arrival times are unpredictable and part of them are exactly known in advance. Already existing jobs J_i and just-arrived jobs J_{ai} form job agents. Part of jobs PJ_i, whose arrivals are forecasted to occur within the time horizon L, form pseudo-job-agent. All of the job agents and pseudo-job-agent participate in auctions beginning at decision-point t_c.

D. Good Forecast of Job Arrivals

In good forecast of job arrivals case (Fig. 7), all job arrivals are known in advance with a degree of uncertainty. Already existing jobs J_i and just-arrived jobs J_{ai} form job agents. Jobs PJ_i that are forecasted to arrive within time-horizon L form

pseudo-job-agent. All of them participate in auctions beginning at decision-point t_c.

Here, chance constrained programming [32] is used to make good forecast of job-arrival times. In CMAF, job-arrival times are the same as job-release dates. In this case, variables of job-release dates r_i are viewed as stochastic variables. Constraints (7) become probabilistic constraints (15), as shown later. This means that constraints are not expected to be always satisfied, but will only be satisfied with given probabilities. Using chance constrained programming, the probabilistic constraints (15) can be converted into their respective deterministic equivalents (16) according to the predetermined confidence levels (α_i. In the deterministic constraints (16), expected job-release dates r_i are replaced with their anticipated job-release dates R_i (17) that are in terms of the means and variances of stochastic variables r_i. Thus, there is a parameter value defining the degree of available information of variances. More variance information would make a better forecast and, hence, a better predictive decision-making.

The probabilistic constraints and their equivalent deterministic constraints are given by

$$\text{Prob}\left[\sum_{t=t_c}^{t_c+L} tX_{i1t} - p_{i1} \geq r_i\right] \geq \alpha_i, \quad 0 \leq \alpha_i \leq 1 \tag{15}$$

$$\sum_{t=t_c}^{t_c+L} tX_{i1t} \geq p_{i1} + R_i \tag{16}$$

$$R_i = F_i^{-1}(\alpha_i) \tag{17}$$

$$\text{Prob}\left[r_i \leq z\right] = F_i(z) \tag{18}$$

where $i = 1, \ldots, N_c$ and $F_i(.)$ is the distribution function of stochastic variable r_i.

V. Experimental Design for Dynamic Job Arrivals

Computational experiments are designed and implemented to demonstrate the effectiveness and robustness of the framework under unpredictable job arrivals and the effects of making full use of available uncertain information of disruptions upon the framework for further scheduling robustness enhancement.

A. Benchmark Problems

First, five instances are used in the experiments of static job shop scheduling. They are Qi8x4 from [33]; Muth6*6, Muth10*10, and Muth20*5 from [34]; and Trentesaux30x3 from [35].

LEKIN software [36] is used to give the performance of several commonly used dispatching rules, which include weighted shortest processing time (WSPT), minimum slack (MS), critical ratio (CR), first come and first served (FCFS), earliest due date (EDD), shortest processing time (SPT), and longest processing time (LPT). LEKIN is an educational scheduling system developed at the Stern School of Business, New York University. It only provides static job shop scheduling. But for nonzero job arrivals, static job shop scheduling and dynamic job shop scheduling with dynamic job arrivals have the same results under these seven dispatching rules. Thus, the results of LEKIN can be used for comparison in dynamic job shop scheduling with dynamic job arrivals.

In all of the five instances that are considered, the jobs have zero release dates. As these instances were formulated originally to measure makespan, a weight and a due date are added for each job in order to measure the total weighted tardiness. Weights are randomly generated using a uniform distribution, U(1, 10). Due dates are set according to the following formula [37]:

$$d_i = r_i + \left\lfloor f \sum_{j=1}^{O_i} p_{ij} \right\rfloor \tag{19}$$

where f is a due-date tightness factor.

Second, the same five instances are also used in the experiments of dynamic job shop scheduling. In order to measure the total weighted tardiness for dynamic job shop scheduling, a weight, a due date, and a nonzero release date are added for each job. Weights and due dates are generated similarly to the static case described earlier. The job-arrival types for partial perfect forecast of job arrivals case are generated randomly. The generation of release dates is presented in Section V-B.

B. Forecast

In the experiments of good forecast of job arrivals' case, stochastic variable r_i, representing expected release date for job i, is assumed to have a uniform distribution $U(a_i, b_i)$. The value used in the simulation is equal to the mean of the stochastic variable. Let

$$a_i = r_i - \beta_i \cdot r_i \tag{20}$$

$$b_i = r_i + \beta_i \cdot r_i \tag{21}$$

where β_i is defined as an uncertainty level of release date,$0 \leq \beta_i \leq 1$. Therefore

$$r_i = \frac{a_i + b_i}{2}. \tag{22}$$

r_i is also the mean of $U(a_i, b_i)$. According to (16), anticipated release date R_i and actual release date r_i' are derived from

$$R_i = a_i + \alpha_i(b_i - a_i) \tag{23}$$

$$r_i' = a_i + U(0,\ 1)(b_i - a_i). \tag{24}$$

Then

$$R_i = r_i + 12 \times \frac{\alpha_i - 0.5}{b_i - a_i}\sigma_i \tag{25}$$

where σ_i is the variance of $U(a_i, b_i)$. Therefore, anticipated release date R_i is in terms of the mean r_i and the variance σ_i of stochastic variable r_i.

Three values of α_i are considered (0.2, 0.5, and 0.8), which means the levels of variances (negative variance, no variance, and positive variance). In addition, two values of β_i are considered (0.2 and 0.8), which means the levels of uncertainty (low uncertainty and high uncertainty).

The relationships among these parameters of good forecast are shown in Fig. 8.

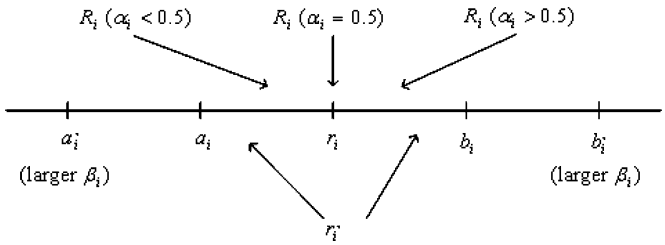


Fig. 8. Relationships among parameters of good forecast.

This relationship is described as follows: expected release date r_i has a value which is the mean of $U(a_i, b_i)$. Anticipated release date R_i is a good forecast of the actual release date r_i'. There are variances in R_i when it is less than the mean (i.e., when $\alpha_i > 0.5$) or greater than the mean (i.e., when $\alpha_i < 0.5$). There is no variance in R_i when it is equal to the mean (i.e., when $\alpha_i = 0.5$). Actual release date r_i' is randomly generated within $U(a_i, b_i)$, which has a larger range with a larger β_i representing the uncertainty level of release date. In the experiments, α and β are used to represent α_i's and β_i's, respectively.

In the two instances Qi8*4 and Muth6*6, three jobs are grouped together and have the same expected release date, which is assigned as 3, 6, or 9. In the instances Muth10*10, Muth20*5, and Trentesaux30x3, first one job, then two jobs, and finally three jobs are grouped together, respectively, and are assigned r_i as 3, 13, or 23, etc. Due to the random generation of actual release dates r_i', jobs do not actually arrive at the same time.

In summary, on the forecast of job arrivals, in perfect forecast and partial perfect forecast cases, pseudo-job-agent use actual release dates r_i' in the predictive decision-making procedure; in good forecast case, pseudo-job-agent use anticipated release date R_i.

C. Computational Complexity

The simulation is implemented in Microsoft Visual C++ 6.0 with an object-oriented programming approach.

Computational complexity for final schedule's construction of a machine agent is n^* *O* (*one auction of a machine agent*), where n represents the total number of jobs in a long run. As the framework is for dynamic job shop scheduling, more attention is paid on the computational time of getting a schedule for an available machine agent at the current decision point. Therefore, its computational complexity is *O (one auction of a machine agent)*, which is equal to *the number of price-adjusting iterations * O (computing bids of a job agent)*. Furthermore, it equals *O (computing bids of a job agent)*.

In the simulation, problem, $P_{\lambda i}$ is transferred into an assignment problem with additional constraints, which is named as problem $AP_{\lambda i}$. The Hungarian algorithm [38] is adapted to solve the $AP_{\lambda i}$. So *O (computing bids of a job agent)* is equal to *O (mL)*, where m represents the total number of machines in the long run.

Therefore, computational complexity is *O(mL)* for obtaining an immediate schedule for an available machine agent at the current decision point.

VI. Computational Results and Analysis

This section presents the results of computational experiments and analysis in detail. First, it compares computational results on unpredictable job arrivals among the presented framework and the commonly used dispatching rules to show the effectiveness and robustness of the framework. Second, it compares computational results among four cases of dynamic job arrivals to demonstrate the effects of making full use of available uncertain information of disruptions. Then, two examples are given for validation of results. Finally, we consider the sensitivity analysis of the length of rolling time horizon L.

A. Static Job Shop Scheduling

Table I shows the performance of the five instances for static job shop scheduling because all jobs have zero release dates. Each data is an averaged one over five runs.

Here, "*" means that this value is better than CMAF's. Comparing CMAF with dispatching rules, the sequence of the average ranking among dispatching rules and CMAF is 2.3 (WSPT), 3.1 (CMAF), 3.4 (EDD), 3.5 (SPT), 4.5 (MS), 4.8 (CR), 6.0 (FCFS), and 7.6 (LPT). Using the following formula for ranking percent deviation (26), WSPT has 62% ranking-percent deviation and CMAF has 43%.

The ranking percent deviation equals

$$\frac{\sum |(\text{a ranking}) - (\text{the average ranking})|}{(\text{the number of data columns}) \times (\text{the average ranking}).} \tag{26}$$

The ranking-percent deviation is the measure of how close the measured rankings are to the average ranking. The greater the percent deviation, the farther the rankings are to the average ranking, which also means there is more rise and fall, i.e., fluctuating performance. This shows that CMAF works more stably than WSPT. Thus, CMAF has very good and stable performance for static job shop scheduling.

B. Dynamic Job Shop Scheduling

In the experiments for dynamic job shop scheduling, $f = 1.7$ is used in Qi8*4, Muth6*6, and Muth10*10 instances; $f = 2$ is used in Muth20*5 and Trentesaux30*3 instances. Tables II–VI show the performance with Qi8*4, Muth6*6, Muth10*10, Muth20*5, and Trentesaux30*3 for dynamic job shop scheduling, respectively. Datas are averaged one over five runs. In these tables, (X, Y) means that $\alpha = X$ and $\beta = Y$. CMAF-U represents unpredictable job arrivals case of CMAF; CMAF-PE is the perfect forecast of job arrivals case of CMAF; CMAF-PA is the partial perfect forecast of job arrivals case of CMAF; and CMAF-G is the good forecast of job arrivals case of CMAF. "*" means that this value is better than CMAF-U's.

Comparing CMAF-U with dispatching rules, the sequence of the average ranking among dispatching rules and CMAF-U is 2.5 (WSPT), 3.2 (CMAF), 3.6 (MS), 3.8 (SPT), 4.1 (EDD), 4.7 (CR), 5.9 (FCFS), and 7.3 (LPT). But WSPT has 55% ranking-percent deviation and CMAF has 45%. From these, it shows that

TABLE I
PERFORMANCE FOR STATIC JOB SHOP SCHEDULING

(a)

LEKIN®	Qi8*4 ($\Sigma w_i T_i$)				Muth6*6 ($\Sigma w_i T_i$)				Muth10*10 ($\Sigma w_i T_i$)	
	f=1.3	F=1.5	f=1.7	f=2	f=1.3	f=1.5	f=1.7	f=2	f=1.3	f=1.5
CR	471	344	188	28*	467	194	58	22	14690*	10161
EDD	409	326	283	110	277	100*	36*	0*	13312*	8299*
FCFS	422	265	134*	51	400	296	199	44	18892	10785
LPT	561	437	277	106	505	354	278	160	25462	16021
MS	515	288	145*	14*	380	114	23*	0*	16609	9279
SPT	391	325	275	119	226	110	55	37	12752*	7374*
WSPT	320	252	199	103	201	92*	69	61	12320*	7398*
CMAF	305	218	156	43	187	110	48	22	15977	8444

(b)

LEKIN®	Muth10*10 ($\Sigma w_i T_i$)			Muth20*5 ($\Sigma w_i T_i$)			Trentesaux30*3 ($\Sigma w_i T_i$)			
	f=1.7	f=2	f=1.3	f=1.5	f=1.7	f=2	f=1.3	f=1.5	f=1.7	f=2
CR	7111	3989	47757*	53835	46956*	39162	48369	49671	46416	41754
EDD	2680*	261*	48470*	46295*	41062*	21878*	48039	46896	45916	39220
FCFS	8208	3328	101006	93983	98390	79976	63181	73284	67087	60778
LPT	13440	7812	100454	96816	96611	81808	74524	81455	72730	67041
MS	4919*	9*	68525	64906	55530	42112	69602	72063	64185	55133
SPT	2906*	124*	46873*	45611*	40291*	29013*	48678	50600	47032	40509
WSPT	4326*	1767*	37296*	34312*	31869*	19557*	37424*	38311*	37044*	22896*
CMAF	6254	2201	55927	50246	48438	35107	45251	45602	42844	39193

TABLE II
PERFORMANCE WITH QI8*4 FOR DYNAMIC JOB SHOP SCHEDULING

LEKIN®	(0.2, 0.2) $\Sigma w_i T_i$	(0.2, 0.8) $\Sigma w_i T_i$	(0.5, 0.2) $\Sigma w_i T_i$	(0.5, 0.8) $\Sigma w_i T_i$	(0.8, 0.2) $\Sigma w_i T_i$	(0.8, 0.8) $\Sigma w_i T_i$
CR	234	224	202	201	240	258
EDD	238	286	254	228	306	289
FCFS	142	149	124	181	140	188
LPT	214	190	192	210	278	208
MS	207	157	184	180	202	180
SPT	298	267	324	233	245	312
WSPT	163	156	137	157	186	186
CMAF-U	139	138	96	147	129	155
CMAF-PE	138	148	109	148	130	156
CMAF-PA	138	138	96	147	130	155
CMAF-G	133	93	117	102	140	158

TABLE III
PERFORMANCE WITH MUTH6*6 FOR DYNAMIC JOB SHOP SCHEDULING

LEKIN®	(0.2, 0.2) $\Sigma w_i T_i$	(0.2, 0.8) $\Sigma w_i T_i$	(0.5, 0.2) $\Sigma w_i T_i$	(0.5, 0.8) $\Sigma w_i T_i$	(0.8, 0.2) $\Sigma w_i T_i$	(0.8, 0.8) $\Sigma w_i T_i$
CR	47*	125	62*	31*	77*	103
EDD	59*	69*	49*	54	54*	55*
FCFS	136	157	127	96	126	135
LPT	259	270	257	187	269	278
MS	10*	20*	13*	24*	16*	67
SPT	59*	113	49*	54	54*	130
WSPT	69	99	94	54	102	53*
CMAF-U	66	94	74	36	90	66
CMAF-PE	66	114	89	57	101	66
CMAF-PA	66	94	74	36	90	66
CMAF-G	73	112	73	72	90	62

TABLE IV
PERFORMANCE WITH MUTH10*10 FOR DYNAMIC JOB SHOP SCHEDULING

LEKIN®	(0.2, 0.2) $\Sigma w_i T_i$	(0.2, 0.8) $\Sigma w_i T_i$	(0.5, 0.2) $\Sigma w_i T_i$	(0.5, 0.8) $\Sigma w_i T_i$	(0.8, 0.2) $\Sigma w_i T_i$	(0.8, 0.8) $\Sigma w_i T_i$
CR	6658	7070	4639	6250	6613	6887
EDD	4505*	2807*	3077*	2160*	3517*	3011*
FCFS	7847	8211	6466	7800	6093	8199
LPT	13896	12234	11020	10638	11793	12780
MS	2835*	1349*	1482*	1521*	2245*	1942*
SPT	3098*	2010*	2504*	2031*	2809*	2124*
WSPT	5585*	3353*	3602*	2678*	4052*	2322*
CMAF-U	5925	4136	3832	4587	4331	4646
CMAF-PE	5925	4136	3832	4587	4331	4646
CMAF-PA	5925	4136	3832	4587	4331	4646
CMAF-G	5389	4763	3620	5828	4328	4646

TABLE V
PERFORMANCE WITH MUTH20*5 FOR DYNAMIC JOB SHOP SCHEDULING

LEKIN®	(0.2, 0.2) $\Sigma w_i T_i$	(0.2, 0.8) $\Sigma w_i T_i$	(0.5, 0.2) $\Sigma w_i T_i$	(0.5, 0.8) $\Sigma w_i T_i$	(0.8, 0.2) $\Sigma w_i T_i$	(0.8, 0.8) $\Sigma w_i T_i$
CR	37543*	35888	37689	32463*	35910*	34796*
EDD	30771*	29087*	30202*	26996*	29528*	28905*
FCFS	79061	75204	78062	76465	81685	74314
LPT	91567	86311	93978	82515	92001	84859
MS	43286	37508	41054	36954	40916	39760
SPT	31945*	26235*	28673*	25046*	30226*	28745*
WSPT	19286*	21829*	22184*	22136*	21350*	21669*
CMAF-U	40111	35623	34479	33109	40123	38856
CMAF-PE	40111	35623	34479	33109	40123	38856
CMAF-PA	40111	35623	34479	33109	40123	38856
CMAF-G	41315	34155	35957	32341	40123	38775

TABLE VI
PERFORMANCE WITH TRENTESAUX30*3 FOR DYNAMIC JOB SHOP SCHEDULING

LEKIN®	(0.2, 0.2) $\Sigma w_i T_i$	(0.2, 0.8) $\Sigma w_i T_i$	(0.5, 0.2) $\Sigma w_i T_i$	(0.5, 0.8) $\Sigma w_i T_i$	(0.8, 0.2) $\Sigma w_i T_i$	(0.8, 0.8) $\Sigma w_i T_i$
CR	41252	44521	42818	43055	41605	36653
EDD	41636	45153	43322	47132	39908	36873
FCFS	51104	61687	53365	60898	49918	52110
LPT	64191	70474	69205	69223	63274	61963
MS	42856	47069	44152	49813	41370	39534
SPT	37526	45929	42250	43367	39460	36141
WSPT	27743*	32996*	29913*	30951*	26748*	26021*
CMAF-U	36358	42408	38298	42696	34165	35519
CMAF-PE	33949	40467	36497	41359	34228	34936
CMAF-PA	36358	42430	36738	42487	34165	35080
CMAF-G	34674	41559	37012	42857	33301	34651

TABLE VII
PERFORMANCE CHANGING BY USING AVAILABLE UNCERTAIN INFORMATION

Use of available uncertain information	Performance of unpredictable job arrivals case			
	Changed			Unchanged
	Total	Worse	Better	
Perfect forecast	30.70%	13.35%	17.35%	69.30%
Partial perfect forecast	6.70%	1.35%	5.35%	93.30%
Good forecast	60.70%	26.70%	34.00%	39.30%
(0.2, 0.2)	13.30%	7.30%	6.00%	3.30%
(0.2, 0.8)	14.70%	4.70%	10.00%	2.00%
(0.5, 0.2)	10.70%	4.00%	6.70%	6.00%
(0.5, 0.8)	10.70%	6.00%	4.70%	6.00%
(0.8, 0.2)	4.70%	2.00%	2.70%	12.00%
(0.8, 0.8)	6.70%	2.70%	4.00%	10.00%

TABLE VIII
MUTH10*10 (0.2, 0.2) EXAMPLE

Job i	Weight w_i	Due date d_i	Machine (processing time) m_{ij} (p_{ij})	Expected release date r_i	Anticipated release date R_i	Actual release date r_i'	Job type
1	4	674	1 (29), 2 (78), 3 (9), 4 (36), 5 (49), 6 (11), 7 (62), 8 (56), 9 (44), 10 (21)	3	2	2	0
2	9	880	1 (43), 3 (90), 5 (75), 10 (11), 4 (69), 2 (28), 7 (46), 6 (46), 8 (72), 9 (30)	13	11	15	1
3	5	988	2 (91), 1 (85), 4 (49), 3 (74), 9 (90), 6 (10), 8 (12), 7 (89), 10 (45), 5 (33)	23	20	22	0
4	9	1146	2 (81), 3 (95), 1 (71), 5 (99), 7 (9), 9 (52), 8 (85), 4 (98), 10 (22), 6 (43)	33	29	38	1
5	6	711	3 (14), 1 (6), 2 (22), 6 (61), 4 (26), 5 (69), 9 (21), 8 (49), 10 (72), 7 (53)	43	38	43	1
6	10	896	3 (84), 2 (2), 6 (52), 4 (95), 9 (48), 10 (72), 1 (47), 7 (65), 5 (6), 8 (25)	53	46	63	1
7	1	770	2 (46), 1 (37), 4 (61), 3 (13), 7 (32), 6 (21), 10 (32), 9 (89), 8 (30), 5 (55)	63	55	51	0
8	5	989	3 (31), 1 (86), 2 (46), 6 (74), 5 (32), 7 (88), 9 (19), 10 (48), 8 (36), 4 (79)	73	64	71	0
9	9	1097	1 (76), 2 (69), 4 (76), 6 (51), 3 (85), 10 (11), 7 (40), 8 (89), 5 (26), 9 (74)	83	73	96	1
10	9	1011	2 (85), 1 (13), 3 (61), 7 (7), 9 (64), 10 (76), 6 (47), 4 (52), 5 (90), 8 (45)	93	82	107	1

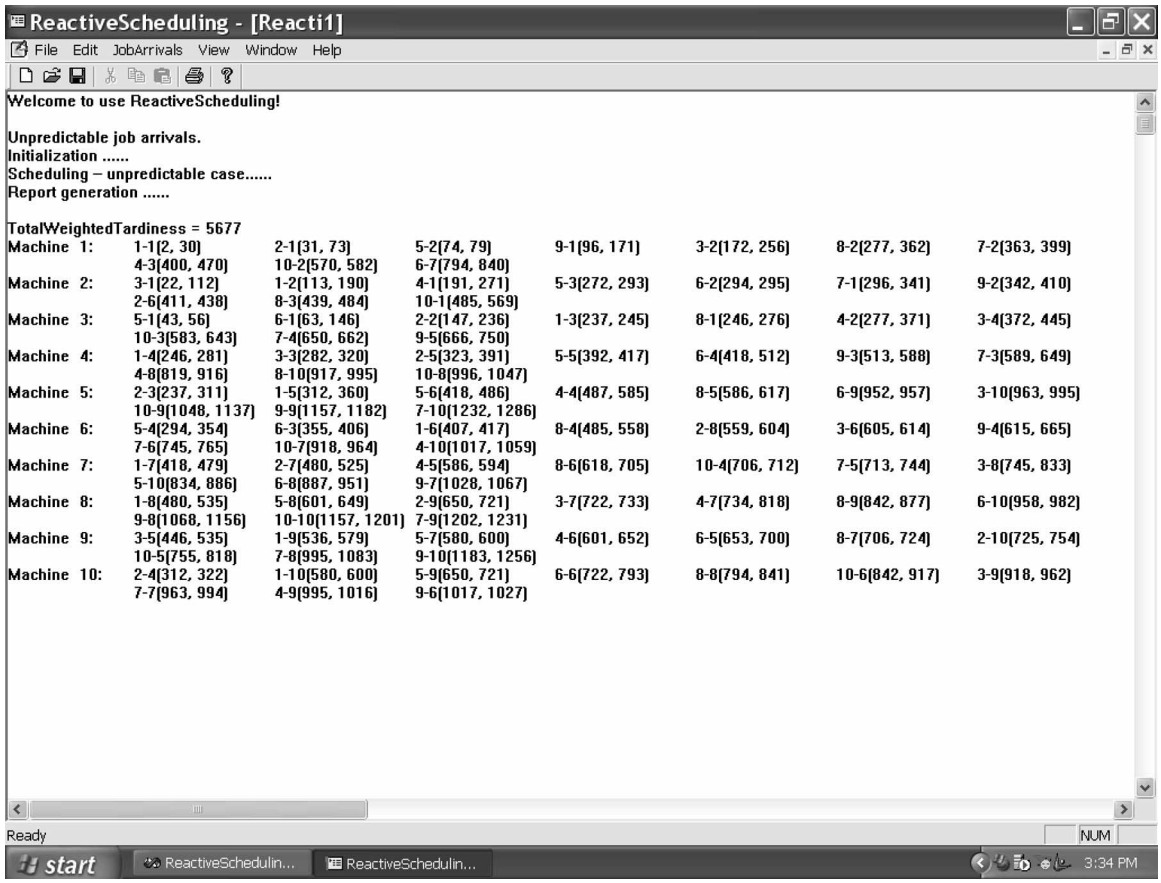


Fig. 9. Schedule obtained by CMAF-U for Muth10*10 (0.2, 0.2) example.

TABLE IX
15×4 JOB SHOP EXAMPLE

Job i	Release date r_i	Weight w_i	Due date d_i	Machine (processing time) m_{ij} (p_{ij})
1	6	55	63	2 (8), 1 (8), 4 (27), 3 (7)
2	9	64	66	3 (6), 4 (14), 1 (13), 2 (16)
3	12	42	69	3 (5), 4 (14), 2 (6), 1 (28)
4	15	57	72	1 (8), 2 (8), 3 (7), 4 (18)
5	18	40	75	2 (5), 4 (27), 3 (4), 1 (14)
6	21	60	78	1 (14), 3 (7), 4 (8), 2 (8)
7	24	58	81	3 (4), 1 (10), 2 (5), 4 (21)
8	27	42	84	2 (6), 1 (10), 3 (5), 4 (21)
9	30	49	87	3 (7), 1 (9), 4 (13), 2 (5)
10	33	59	90	3 (4), 4 (10), 1 (11), 2 (10)
11	36	51	93	1 (13), 2 (5), 3 (7), 4 (27)
12	39	53	96	3 (5), 2 (5), 1 (11), 4 (21)
13	42	68	99	3 (7), 4 (14), 2 (14), 1 (8)
14	45	50	102	2 (6), 1 (11), 4 (21), 3 (15)
15	48	69	105	1 (13), 4 (21), 2 (8), 3 (16)

CMAF has also very good and stable performance for dynamic job shop scheduling with unpredictable job arrivals.

Although WSPT is ranked higher than CMAF dealing with unpredictable job arrivals, there are several reasons for researching on CMAF. First of all, CMAF is more stable than WSPT due to its ranking-percent deviations. Second, WSPT is created and used in a centralized model with global information of all jobs' weights and processing times. However, in CMAF aimed at minimum global information, agents are independent and the global information is only the numbers of current jobs and machines. Third, CMAF is exploring an approach with a purely distributed model, auctions, and multiple agents aimed at no master/slave relationships for dynamic job shop scheduling and job shop scheduling for distributed manufacturing systems. Last, as WSPT uses processing times to select jobs, its performance in dynamic job shop scheduling with processing time variations would be unpredictable.

C. Use of Available Uncertain Information

If using the result of an instance obtained by CMAF-U as a reference, the result of the same instance obtained by CMAF-PE, CMAF-PA, or CMAF-G can fall into two categories: changed (i.e., affected) (among it, there is getting worse effect or getting better effect) or unchanged (i.e., unaffected) with the forecast of job arrivals when comparing with the one by CMAF-U. For example, there are four results for one instance: 55 (CMAF-U), 110 (CMAF-PE), 55 (CMAF-PA), and 40 (CMAF-G). CMAF-PE and CMAF-G change the result of CMAF-U, but CMAF-PA does not. CMAF-PE has a worse effect in this instance while CMAF-G has a better one, but CMAP-PA has no effect in this instance.

Comparing CMAF-U with CMAF-PE, only 30.7% of instances are affected by CMAF-PE. Among affected instances, 13.35% have worse performance and 17.35% have better performance with perfect forecast of job arrivals. However, 69.3%

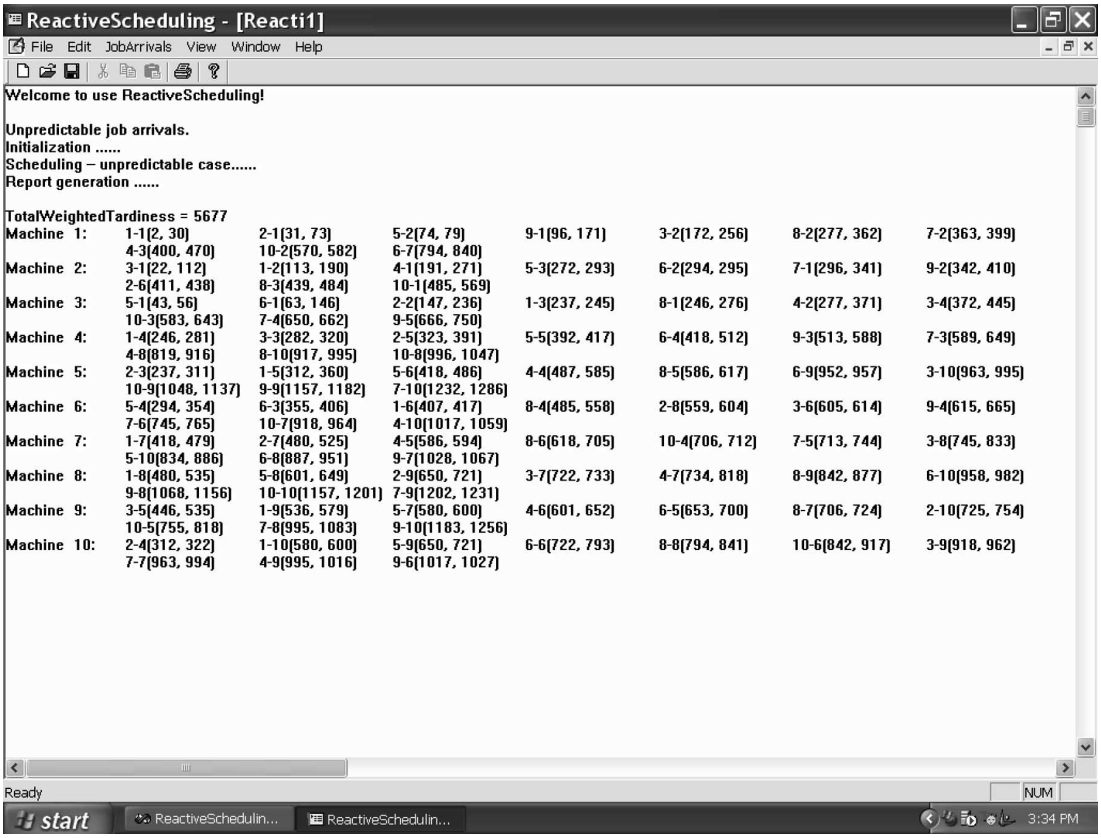


Fig. 10. Schedule obtained by CMAF-U for 15 × 4 job shop example.

of instances are unaffected. This shows that perfect forecast of job arrivals has very few effects on the framework.

Comparing CMAF-U with CMAF-PA, only 6.7% of instances are affected by CMAF-PA. Among them, 1.35% have worse performance, while 5.35% have better performance. Significantly, 93.3% of instances are unaffected by CMAF-PA. Therefore, partial perfect forecast of job arrivals has few effects on the framework.

According to the definition of scheduling robustness, CMAF works mostly as well as CMAF-PE and CMAF-PA under unpredictable job arrivals. Therefore, the framework works robustly in dynamic job shop scheduling with unpredictable job arrivals.

Comparing CMAF-U with CMAF-G, 60.7% of instances are affected by CMAF-G. Among them, 26.7% have worse performance, but 34.0% have better performance. Significantly, 39.3% of instances are unaffected by CMAF-G. This shows that good forecast of job arrivals has few effects on the framework.

Table VII is the summary of the above information and the percentage distribution among the combinations of α and β values in good-forecast case.

From Table VII, it is clear that making full use of available uncertain information of disruptions has some effects on the framework for further enhancement of scheduling robustness. Good forecast brings much more effects than perfect forecast and partial perfect forecast. Among the effects of the three forecast cases, although there are worse and better effects, the better effects are always more than the worse effects. For example, for perfect forecast in Table VII, there are 13.35% worse effects compared to 17.35% better effects. It also shows that when there is variance, the high level of uncertainty brings more and better effects than the low level of uncertainty. However, when there is no variance, the low level of uncertainty brings more and better effects than the high level of uncertainty. In addition, under high levels of uncertainty, positive and negative variances work better than no variance because the former brings less worse effects.

1) Examples: The data shown in Table VIII is an example of Muth10*10 (0.2, 0.2). In "Job type" column, "0" and "1" mean perfect forecast job type and unpredictable job type, respectively. The column "Actual release date" is used in both CMAF-U and CMAF-PE. The columns "Job type" and "Actual release date" are used in CMAF-PA. The columns "Anticipated release date" and "Actual release date" are used in CMAF-G. The column "Expected release date" is used to generate columns "Anticipated release date" and "Actual release date." The results of CMAF-U, CMAF-PE, CMAF-PA, and CMAF-G are 5677, 5677, 5677, and 5696. The schedule obtained by CMAF-U is shown in Fig. 9.

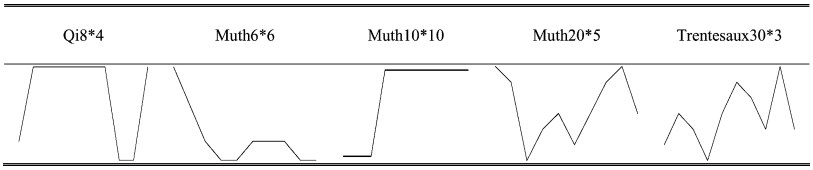
The data shown in Table IX is a practical modified 15 × 4 job shop example from [25]. The types of operations are milling, drilling, and boring. The schedule obtained by CMAF-U is shown in Fig. 10.

In Figs. 9 and 10, the format "x1-x2 (y1, y2)" means, "job no.-operation no. (beginning processed time, finishing processed time)." For the results validation, the obtained schedule

TABLE X
EFFECTS OF THE LENGTH OF L ON PERFORMANCE OF CMAF-U

		$\Sigma w_i T_i$									
	L	+0	+10	+20	+30	+40	+50	+60	+70	+80	+90
Qi8*4	10	136	180	180	180	180	180	180	133*	133	180
Muth6*6	10	252	198	117	54	54*	117	117	117	54	54
Muth10*10	100	5668	5668	5668*	6448	6448	6448	6448	6448	6448	6448
Muth20*5	100	58063	56874	51392*	51726	54627	51554	53939	56628	58384	53950
Trentesaux30*3	50	41922	41544	41693	41699	38904	41189	41345	41069	35487*	39516

TABLE XI
PLOTS OF EFFECTS OF L ON PERFORMANCE



is checked out by constraints such as release dates, machines (processing times), operation precedence, and machine capacity. When checking constraints of release dates, it is to check if the beginning processed times of the first operations of jobs are equal or greater than the release dates. For example, in Fig. 10, the beginning processed time of no. 15 job is 163, while its release date is 48. Checking machines (processing times) is to check if the processed operations of every job satisfy the job's description. For example, the processed operation 3 of no. 1 job in Fig. 10 is 1-3 (29, 55). The 1-3 operation is processed on Machine 4 and its processing time is 27; hence, as its description is 4 (27). Checking constraints of operation precedence is to check if the operations of a job are in order as described. For example, no. 1 job in Fig. 10 has the ordered sequence: 1-1 (6, 13), 1-2 (14, 21), 1-3 (29, 55), and 1-4 (60, 66). Checking constraints of machine capacity is to check if any machine has only one operation being processed at any time. For example, the schedule of Machine 1: 1-2 (14, 21), 4-1 (22, 29), 2-3 (30, 42), 6-1 (43, 56), etc. in Fig. 10 has only one operation on the machine at any time.

D. Sensitivity Analysis of the Length of L

The problem size of a job shop scheduling problem is $n * m$, where n is the total number of jobs in a long run and m is the total number of machines. The computational complexity is *n*O(mL)* in a long run, where *O (mL)* is the computational complexity of one auction. Due to the matrix computation used, the length of rolling time horizon must be selected greater than m and the longest processing time. However, L should not be so long as to increase computational times.

In the experiments of sensitivity analysis of the length of L, instances (0.2, 0.2) in Qi8*4 and Muth6*6, instance (0.8, 0.8) in Muth10*10, and instances (0.2, 0.8) in Muth20*5 and Trentesaux30*3 are used. As L must be greater than m and the longest processing time, the following initial values of L are selected; 10 is for Qi8*4 and Muth6*6, 100 is for Muth10*10 and Muth20*5, and 50 is for Trentesaux30*3. Table X shows the effects of the length of L on the performance of unpredictable job arrivals' case.

In Table X, values marked with "*" mean that the corresponding L provides the best performance. Table XI shows the corresponding plots of data of Table X. The lowest points on the plots represent the best performance in each case. The table shows that the selection of L for the best performance depends on the case being simulated.

VII. CONCLUSION

In this paper, the theoretical basis and experimental justification of the arguments about a complete multiagent framework for dynamic job shop scheduling considering robustness and adaptability are presented. Computational experiments on dynamic job arrivals are provided for the experimental justification.

From the computational results, it can be concluded that the framework works very good and stably in both static job shop scheduling and dynamic job shop scheduling with unpredictable job arrivals, and robustly in dynamic job shop scheduling with unpredictable job arrivals. Making full use of available uncertain information of disruptions brings some effects on the framework for further scheduling robustness enhancement, both worse and better. However, good forecast of job arrivals plays a stronger role in it than perfect forecast and partial perfect forecast of job arrivals. In good forecast, when there is variance, the high level of uncertainty works better than the low level of uncertainty. But, when there is no variance, the low level of uncertainty works better. Under the high level of uncertainty, having variance works better than having no variance. Currently, computational experiments on unpredictable processing time variations and formal verification of the system are being conducted.

REFERENCES

[1] V. Jorge Leon, S. D. Wu, and R. H. Storer, "Robustness measures and robust scheduling for job shops," *IIE Trans.*, vol. 26, no. 5, pp. 32–43, Sep. 1994.

[2] K. Bhaskaran and M. Pinedo, "Dispatching," in *Handbook of Industrial Engineering*, G. Salvendy, Ed. New York: Wiley, 1996, ch. 83.

[3] J. C. Bean, J. R. Birge, J. Mittenthal, and C. E. Noon, "Matchup scheduling with multiple resources, release dates, and disruptions," *Oper. Res.*, vol. 39, no. 3, pp. 470–483, May.–Jun. 1991.

[4] L. K. Church and R. Uzsoy, "Analysis of periodic and event-driven rescheduling policies in dynamic shops," *Int. J. Comput. Integr. Manuf.*, vol. 5, no. 3, pp. 153–163, 1992.

[5] V. J. Leon, S. D. Wu, and R. H. Storer, "A game-theoretic control approach for job shops in the presence of disruptions," *Int. J. Prod. Res.*, vol. 32, no. 6, pp. 1451–1476, 1994.
[6] M. Yamamoto and S. Y. Nof, "Scheduling/rescheduling in the manufacturing operating system environment," *Int. J. Prod. Res.*, vol. 23, no. 4, pp. 705–722, 1985.
[7] E.-S. Byeon, S. D. Wu, and R. H. Storer, "Decomposition heuristics for robust job-shop scheduling," *IEEE Trans. Robot. Autom.*, vol. 14, no. 2, pp. 303–313, Apr. 1998.
[8] S. D. Wu, E.-S. Byeon, and R. H. Storer, "A graph-theoretic decomposition of the job shop scheduling problem to achieve scheduling robustness," *Oper. Res.*, vol. 47, no. 1, pp. 113–124, Jan.–Feb. 1999.
[9] M. Young, *The Techincal Writers Handbook*. Mill Valley, CA: University Science, 1989.
[10] S. V. Mehta and R. M. Uzsoy, "Predictable scheduling of a job shop subject to breakdowns," *IEEE Trans. Robot. Autom.*, vol. 14, no. 3, pp. 365–378, Jun. 1998.
[11] S. V. Mehta and R. Uzsoy, "Predictable scheduling of a single machine subject to breakdowns," *Int. J. Comput. Integr. Manuf.*, vol. 12, no. 1, pp. 15–38, 1999.
[12] S. F. Smith, "Knowledge-based production management: Approaches, results, and prospects," *Prod. Plan. Control*, vol. 3, no. 4, pp. 350–380, 1992.
[13] E. Szelke and R. M. Kerr, "Knowledge-based reactive scheduling," *Prod. Plan. Control*, vol. 5, no. 2, pp. 124–145, 1994.
[14] I. M. Ovacik and R. Uzsoy, "Rolling horizon algorithms for a single-machine dynamic scheduling problem with sequence-dependent setup times," *Int. J. Prod. Res.*, vol. 32, no. 6, pp. 1243–1263, 1994.
[15] W. Shen, "Distributed manufacturing scheduling using intelligent agents," *IEEE Intell. Syst. Appl.*, vol. 17, no. 1, pp. 88–94, Jan.–Feb. 2002.
[16] R. Davis and R. G. Smith, "Negotiation as a metaphor for distributed problem solving," *Artif. Intell.*, vol. 20, no. 1, pp. 63–109, 1983.
[17] R. G. Smith, "The contract net protocol: High-level communication and control in distributed problem solver," *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, Dec. 1980.
[18] G.Y.-J. Lin and J. J. Solberg, "Integrated shop floor control using autonomous agents," *IIE Trans.*, vol. 24, no. 3, pp. 57–71, Jul. 1992.
[19] R. Macchiaroli and S. Riemma, "A negotiation scheme for autonomous agents in job shop scheduling," *Int. J. Comput. Integr. Manuf.*, vol. 15, no. 2, pp. 222–232, 2002.
[20] M. J. Shaw, "A distributed scheduling method for computed integrated manufacturing: The use of local area networks in cellular systems," *Int. J. Prod. Res.*, vol. 25, no. 9, pp. 1285–1303, 1987.
[21] D. M. Upton, "A flexible structure for computer-controlled manufacturing systems," *Manuf. Rev.*, vol. 5, no. 1, pp. 58–72, 1992.
[22] T.-T. Dang and B. Frankovic, "Agent-based scheduling in production systems," *Int. J. Prod. Res.*, vol. 40, no. 15, pp. 3669–3679, 2002.
[23] T. Kawamura, N. Kase, D. Araki, and A. Osuga, "Development of a distributed cooperative scheduling system based on negotiations between scheduling agents," *Syst. Comput. Jpn.*, vol. 31, no. 1, 2000.
[24] J.-S. Liu and K. P. Sycara, "Coordination of multiple agents for production management," *Ann. Oper. Res.*, vol. 75, pp. 235–289, 1997.
[25] J. Váncza and A. Márkus, "An agent model for incentive-based production scheduling," *Comput. Ind.*, vol. 43, no. 2, 2000.
[26] P. Dewan and S. Joshi, "Auction-based distributed scheduling in a dynamic job shop environment," *Int. J. Prod. Res.*, vol. 40, no. 5, pp. 1173–1191, 2002.
[27] E. Kutanoglu and S. D. Wu, "On combinatorial auction and Lagrangean relaxation for distributed resource scheduling," *IIE Trans.*, vol. 31, no. 9, pp. 813–826, 1999.
[28] L. Gou, P. B. Luh, and Y. Kyoya, "Holonic manufacturing scheduling: Architecture, cooperation mechanism, and implementation," *Comput. Ind.*, vol. 37, pp. 213–231, 1998.
[29] P. Dewan and S. Joshi, "Dynamic single-machine scheduling under distributed decision-making," *Int. J. Prod. Res.*, vol. 38, no. 16, pp. 3759–3777, 2000.
[30] T. E. Morton, "Forward algorithms for forward-thinking managers," in *Applications of Management Science,* vol. 1, R. L. Schultz, Ed. Greenwich, CT: JAI, 1981, pp. 1–55.
[31] T. W. Malone and S. A. Smith, "Modeling the performance of organizational structures," *Oper. Res.*, vol. 36, no. 3, pp. 421–435, May–Jun. 1988.
[32] A. Charnes and W. W. Cooper, "Chance-constrained programming," *Manage. Sci.*, vol. 6, pp. 73–79, 1960.
[33] J. G. Qi, G. R. Burns, and D. K. Harrison, "The application of parallel multipopulation genetic algorithms to dynamic job-shop scheduling," *Int. J. Adv. Manuf. Technol.*, vol. 16, pp. 609–615, 2000.
[34] J. Muth and G. Thompson, Eds., *Industrial Scheduling*. Englewood Cliffs, NJ: Prentice-Hall, 1963.
[35] D. Trentesaux, P. Pesin, and C. Tahon, "Comparison of constraint logic programming and distributed problem solving: A case study for interactive, efficient, and practicable job-shop scheduling," *Comput. Ind. Eng.*, vol. 29, pp. 187–211, 2001.
[36] LEKIN, The Stern School of Business, New York University, New York, 2001.
[37] S. Eilon and I. G. Chowdhury, "Due dates in job shop scheduling," *Int. J. Prod. Res.*, vol. 14, no. 2, pp. 223–237, 1976.
[38] G. Carpaneto, S. Martello, and P. Toth, "Algorithms and codes for the assignment problem," *Ann. Oper. Res.*, vol. 13, pp. 193–223, 1988.



Ning Liu (M'04) received the B.S. and M.S. degrees in computer engineering from Zhejiang University, Hangzhou, China, in 1984 and 1987, respectively. She is currently working toward the Ph.D. degree in the Department of Electrical and Computer Engineering, Tennessee Technological University, Cookeville.

From 1987 to 1998, she was an Engineer at the Computer and Information Center, Zhejiang University, Hangzhou. From 1998 to 2001, she was a Senior Engineer at the Networking and Information Center, Zhejiang University. Her current research interests include artificial intelligence, planning, and scheduling.

Ms. Liu is a member of the Association for Computing Machinery.



Mohamed A. Abdelrahman received the B.S. degree in electrical engineering and the M.S. degree in engineering physics from Cairo University, Egypt, in 1988 and 1992, respectively and Cairo, the M.S. degree in measurement and control engineering and the Ph.D. degree in nuclear engineering from Idaho State University, Pocatello, in 1994 and 1996, respectively.

He joined the Department of Electrical and Computer Engineering, Tennessee Technological University (TTU) in 1997, where he is currently an Associate Professor. He is also an Associate Faculty at the Center for Manufacturing Research, TTU. He has been a Principal Investigator for several research projects of the metal casting industry. His current research interests include intelligent systems, measurement systems, mechatronics, and integration of sensing and control with special focus on industrial applications.



Srini Ramaswamy (M'91–SM'00) received the Ph.D. degree in computer science from the University of Southwestern Louisiana, Lafayette (now University of Louisiana, Lafayette), in 1994.

He is currently a Professor and Chairperson of the Department of Computer Science, University of Arkansas at Little Rock, Little Rock. Prior to this, he was the Chairperson of the Department of Computer Science, Tennessee Technological University, Cookeville. During 1994–1995, and subsequently during the summer months of 1996 and 1998, he was a Postdoctoral Research Fellow/Visiting Scientist in the Laboratory for Intelligent Processes and Systems, University of Texas at Austin, Austin. During the summers of 2003 and 2004, he was a Visiting Research Professor of Computer Science at the Institute of Software Integrated Systems, Vanderbilt University, Nashville, TN. His current research interests include intelligent and flexible control systems, behavior modeling, analysis and simulation, software stability and scalability—particularly in the design and development of better software systems for real-time control issues in manufacturing and other distributed, real-time applications.

Prof. Ramaswamy is a member of the Association for Computing Machinery, Society for Computer Simulation International, and Computing Professionals for Social Responsibility.